

程序中的并发 多线程程序设计

主讲老师：高祥



2023/12/12



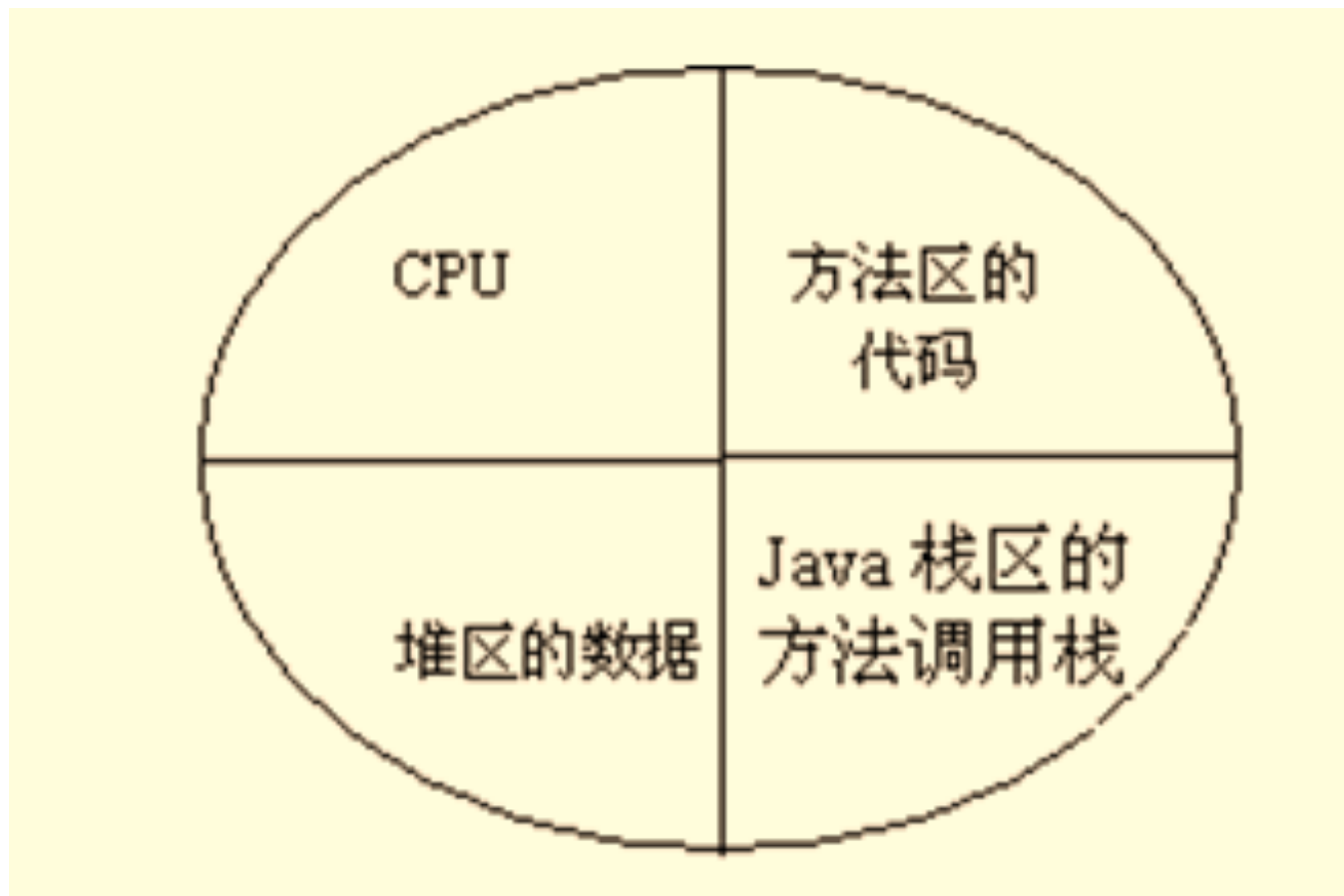
北京航空航天大学
COLLEGE OF SOFTWARE
BEIHANG UNIVERSITY 软件学院

- 进程与线程
- 线程的调度
- Java多线程程序设计
 - 创建和启动线程
 - Thread线程类、
 - Runnable接口
 - 多个线程的同步
 - 线程之间的通信

程序（静态蓝本）和进程（动态执行）

- 程序（program）：是一段静态的代码，是对数据描述与操作的代码的集合，它是应用程序执行的蓝本。
- 进程（process）是程序的一次执行过程，是系统运行程序的基本单位。程序是静态的，进程是动态的。系统运行一个程序即是一个进程从创建、运行到消亡的过程。
- 多任务（multi task）在一个系统中可以同时运行多个程序，即有多个独立运行的任务，每个任务对应一个进程。

线程运行中需要的资源



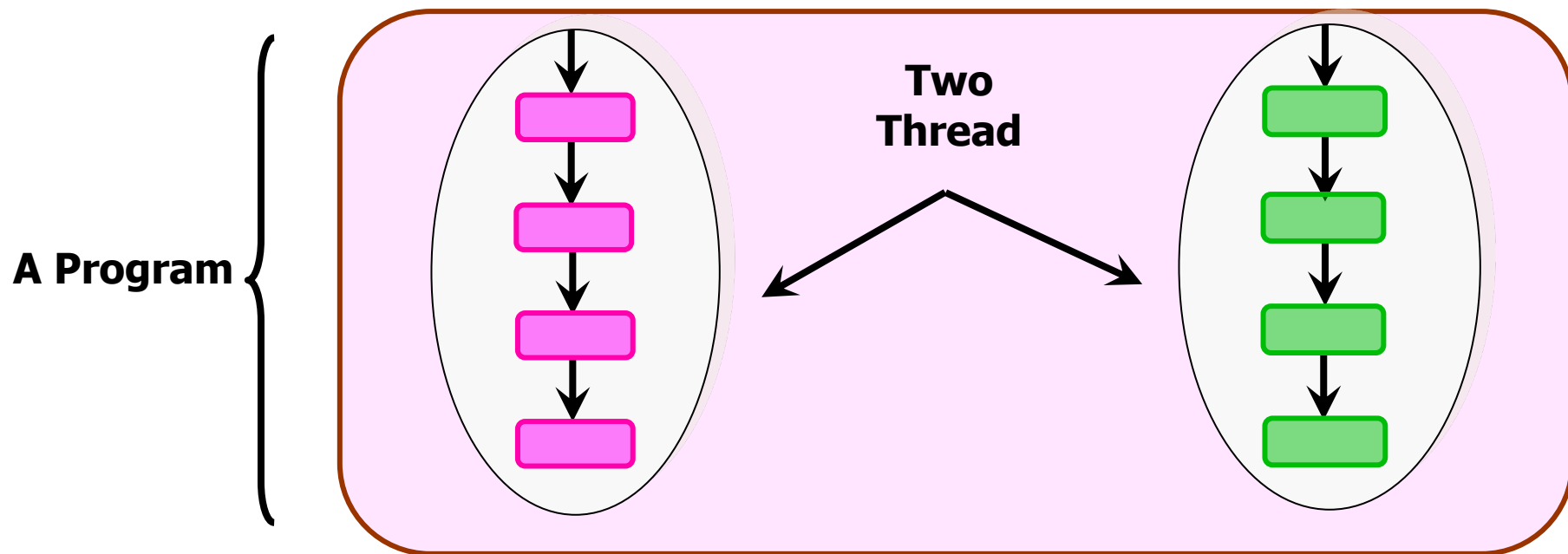
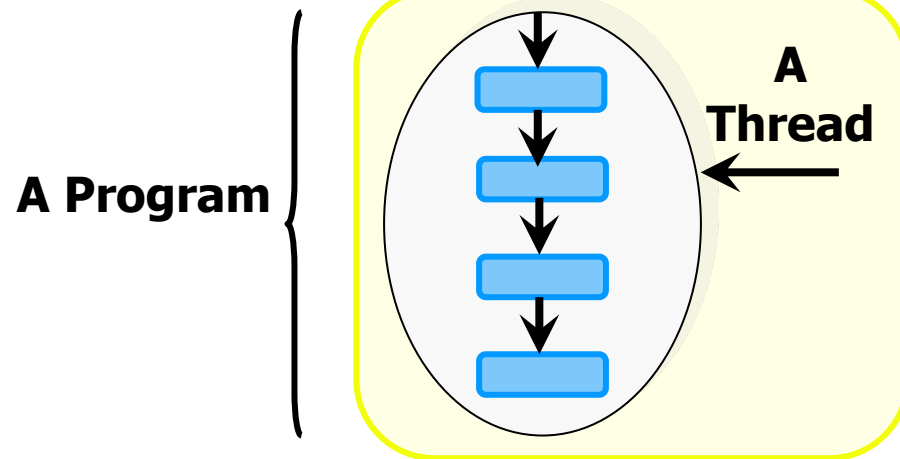
线程

- **线程**：比进程更小的执行单位，一个进程中可以包含多个线程。
 - 一个进程在执行过程中，为了同时完成多项操作，可以产生多个线程，形成多条执行线索。
 - 每个线程都有它自身的产生、存在和消亡的过程。
- 一个进程中的所有线程都在该进程的虚拟地址空间中，**使用该进程的全局变量和系统资源**。
- 操作系统给每个线程分配不同的CPU时间片，**在某一时刻，CPU只执行一个时间片内的线程，多个时间片中的相应线程在CPU内轮流执行**。

线程和进程之间的关系

- **多进程**环境中每一个进程既包括其所要执行的指令，也包括执行指令所需的任何系统资源，如CPU、内存空间、I/O端口等，**不同进程所占用的系统资源相对独立**；
- 线程是比进程单位更小的执行单位，多线程环境中每一个线程都隶属于某一进程，由进程触发执行，在系统资源的使用上，**属于同一进程的所有线程共享该进程的系统资源**；
 - 与进程不同的是线程本身即没有入口，也没有出口，其自身也不能独立运行，它栖身于某个进程之中，由进程启动运行，完成其任务后，自动终止，也可以由进程使之强制终止。

程序、进程和线程示意图



为什么要使用多线程？（系统开销）

- 线程之间共享相同的内存单元(代码和数据)，因此在线程间切换，不需要很大的系统开销，*所以线程之间的切换速度远远比进程之间快，线程之间的通信也比进程通信快的多。*
- 多个线程轮流抢占CPU资源而运行时，从*微观*上讲，一个时间里只能有一个作业被执行，在*宏观*上可使多个作业被同时执行，即等同于要让多台计算机同时工作，*使系统资源特别是CPU的利用率得到提高，从而提高整个程序的执行效率。*

多线程程序设计

- 是指单个程序包含并发执行的多个线程。当多线程程序执行时，该程序对应的进程中就有多个控制流在同时执行，即具有并发执行的多个线程；
- 例如：
 - 影院售票问题
 - Web Server接受客户端的请求问题
 - 银行问题
 - 网络聊天(一对多)程序

- 进程与线程
- 线程的调度
- Java多线程程序设计
 - 创建和启动线程
 - Thread线程类、
 - Runnable接口
 - 多个线程的同步
 - 线程之间的通信

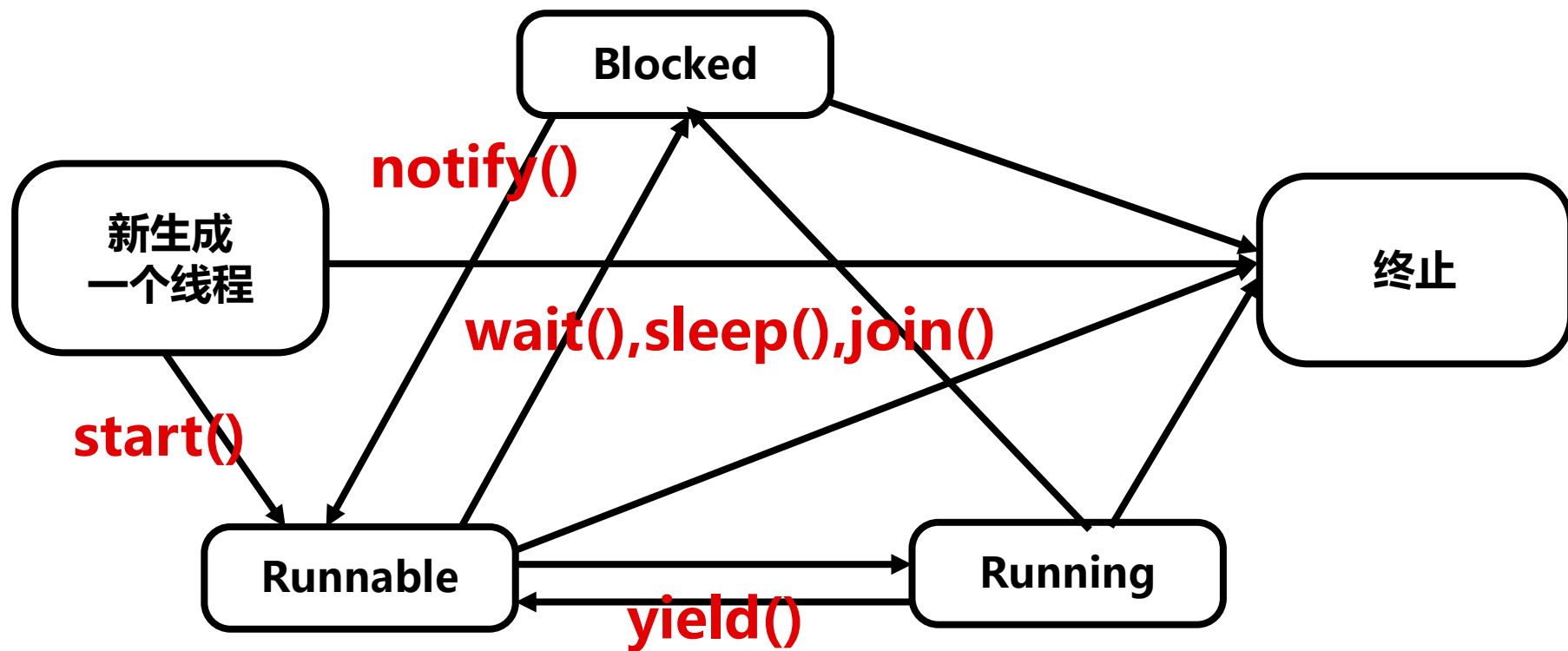
主线程

- `main()` 方法——Application 应用程序
 - 每当用 `java` 命令启动一个 Java 虚拟机进程（Application 应用程序），Java 虚拟机就会创建一个主线程，该线程从程序入口 `main()` 方法开始执行。
- 当在主线程中创建 `Thread` 类或其子类对象时，就创建了一个线程对象。主线程就是上述创建线程的父线程。
- Programmer 可以控制线程的启动、挂起与终止。

线程的状态

- 在一个多线程运行环境中运行的线程可以有多种状态, 也就是说一个线程在不同时刻会处于下列状态之一:
 - 新建、就绪、运行、阻塞、终止;

线程的状态



线程生命周期中的5种状态(1)

- **新建**：当一个 Thread 类或其子类对象被创建时，新产生的线程处于新建状态，此时它已经有了相应的内存空间和其他资源。
如： `Thread myThread=new Thread( )`;
- **就绪**：调用 `start( )` 方法来启动处于新建状态的线程后，将进入线程队列排队等待 CPU 服务，此时它已经具备了运行的条件，一旦轮到它来享用 CPU 资源时，就可以脱离创建它的主线程，开始自己的生命周期。

线程生命周期中的5种状态(2)

- **运行**：当就绪状态的线程被调度并获得处理器资源时，便进入运行状态。
- 每一个 Thread 类及其子类的对象都有一个重要的 run() 方法，当线程对象被调用执行时，它将自动调用本对象的 run() 方法，从第一句开始顺序执行。
- Run() 方法定义了这个线程的操作和功能。

线程生命周期中的5种状态(3)

- 阻塞：一个正在执行的线程暂停自己的执行而进入的状态。引起线程由运行状态进入阻塞状态的可能情况：
 - 该线程正在等待 I/O 操作的完成
 - 网络操作
 - 为了获取锁而进入阻塞操作
 - 调用了该线程的 `sleep()` 方法
 - 调用了 `wait()` 方法
 - 让处于运行状态的线程调用另一个线程的 `join()` 方法

线程生命周期中的5种状态(4)

- 对应于不同进入阻塞状态的情况，采取不同的方法使其回到就绪状态：
 - I/O 操作：等待这个 I/O 操作完成后；
 - sleep() 方法：等待其指定的休眠事件结束后，自动脱离阻塞状态；
 - 调用 wait() 方法：调用 notify() 或 notifyAll() 方法；

线程生命周期中的5种状态(5)

- 终止：
 - 自然终止：线程完成了自己的全部工作
 - 强制终止：在线程执行完之前，调用 `stop()` 或 `destroy()` 方法终止线程

线程的调度

- 微观上，在一台只具有一个CPU的机器上，CPU在同一时间只能分配给一个线程做一件事。
- 当有多于一个的线程工作时，在Java中，线程调度通常是抢占式(即哪一个线程先抢到CPU资源则先运行)，而不是分时间片式。
- 一旦一个线程获得执行权，这个线程将持续运行下去，直到它运行结束或因为某种原因而阻塞，或者有另一个高优先级线程就绪（**这种情况称为低优先级线程被高优先级线程所抢占**）。

线程的调度

- 所有被阻塞的线程按次序排列，组成一个阻塞队列
- 所有就绪但没有运行的线程则根据其优先级排入一个就绪队列。
- 当CPU空闲时，如果就绪队列不空，就绪队列中第一个具有最高优先级的线程将运行。
- 当一个线程被抢占而停止运行时，它的运行态被改变并放到就绪队列的队尾；
- 一个被阻塞（可能因为睡眠或等待I/O设备）的线程就绪后通常也放到就绪队列的队尾。

线程的调度与优先级 (1)

- 线程的调度是按：
 1. 其优先级的高低顺序执行的；
 2. 同样优先级的线程遵循“先到先执行的原则”；

线程的调度与优先级 (1)

- **线程优先级**：范围 $1 \sim 10$ （10 级）。数值越大，级别越高
 - Thread 类定义的 3 个常数：
 - MIN_PRIORITY 最低(小)优先级（值为1）
 - MAX_PRIORITY 最高(大)优先级（值为10）
 - NORM_PRIORITY 默认优先级（值为5）
 - 线程创建时，继承父线程的优先级。
 - 常用方法：
 - `getPriority()`：获得线程的优先级
 - `setPriority()`：设置线程的优先级

主要内容

- 进程与线程
- 线程的调度
- Java多线程程序设计
 - 创建和启动线程
 - Thread线程类
 - Runnable接口
 - 多个线程的同步
 - 线程之间的通信

创建一个新的线程

- Java中的多线程是建立在Thread类, Runnable接口的基础上的, 通常有两种办法让我们来创建一个新的线程:
 - 创建一个Thread类, 或者一个Thread子类的对象;
 - 创建一个实现Runnable接口的类的对象;

线程的创建

- 一个线程的创建肯定是由另一个线程完成的；
- 被创建线程的父线程是创建它的线程；
- 而main线程由JVM创建，而main线程又可以成为其他线程的父线程；

主要内容

- 进程与线程
- 线程的调度
- Java多线程程序设计
 - 创建和启动线程
 - Thread线程类
 - Runnable接口
 - 多个线程的同步
 - 线程之间的通信

应用线程类Thread创建线程

1. 将一个类定义为Thread的子类,那么这个类就可以用来创建线程。
2. 这个类中有一个**至关重要的方法**——**public void run**, 这个方法称为**线程体**, 它是整个线程的**核心**, **线程**所要完成任务的代码都定义在**线程体**中, 实际上**不同功能的线程之间的区别**就在于它们线程体的不同

示例代码

```
public class TimePrinter extends Thread {
```

```
    private int printerId;
```

```
    private int pauseTime;
```

```
    public TimePrinter(int printerId, int pauseTime)
```

```
    {
```

```
        this.printerId = printerId;
```

```
        this.pauseTime = pauseTime;
```

```
    }
```

```
    public void run() {
```

```
        while (true) {
```

```
            try {
```

```
                System.out.println("Printer " + printerId + " is running at " +
```

```
                    new Date(System.currentTimeMillis()));
```

```
                Thread.sleep(pauseTime);
```

```
            } catch (Exception e) {
```

```
                e.printStackTrace();
```

```
            }
```

```
        }
```

```
    }
```

Thread.sleep()方法的使用，注意异常的抛出

运行 结果

```
public static void main(String[] args){  
    TimePrinter tp1 = new TimePrinter(printerId: 1, pauseTime: 1000);  
    TimePrinter tp2 = new TimePrinter(printerId: 2, pauseTime: 3000);  
    tp1.start();  
    tp2.start();  
}
```

```
Printer 1 is running at Wed Dec 07 20:45:15 CST 2022  
Printer 2 is running at Wed Dec 07 20:45:15 CST 2022  
Printer 1 is running at Wed Dec 07 20:45:16 CST 2022  
Printer 1 is running at Wed Dec 07 20:45:17 CST 2022  
Printer 2 is running at Wed Dec 07 20:45:18 CST 2022  
Printer 1 is running at Wed Dec 07 20:45:18 CST 2022  
Printer 1 is running at Wed Dec 07 20:45:19 CST 2022  
Printer 1 is running at Wed Dec 07 20:45:20 CST 2022  
...
```

应用线程类创建线程

创建线程对象

线程子类名 线程对象名
=new 线程子类名 (实际参数)

启动一个线程

线程对象名.start();

线程体的执行

Run()方法的执行

应用线程类Thread创建线程

- Thread的构造方法一共有八个，这里根据命名方式分类，使用默认命名的构造方法如下
 - Thread()
 - Thread(Runnable target)
 - Thread(ThreadGroup group,Runnable target)
- 命名线程的构造方法如下：
 - Thread(String name)
 - Thread(Runnable target,String name)
 - Thread(ThreadGroup group,String name)
 - Thread(ThreadGroup group,Runnable target,String name)
 - Thread(ThreadGroup group,Runnable target,String name,long stackSize)

应用Runnable接口创建线程

1. Runnable是Java中用以实现线程的接口

- **Thread(Runnable target);**
- **Thread(Runnable target, String name);**

2. Runnable接口中只定义了一个方法就是run() 方法，也就是线程体。

示例代码

```
import java.util.*;
class TimePrinter1 implements Runnable {
    int pauseTime;
    String name;
    public TimePrinter1(int x, String n) {
        pauseTime = x;
        name = n;
    }
    public void run() {
        while(true) {
            try {
                System.out.println(name + ":" + new
                    Date(System.currentTimeMillis()));
                Thread.sleep(pauseTime);
            }
            catch(Exception e) {
                System.out.println(e);
            }
        }
    }
    static public void main(String args[]) {
        Thread t1 = new Thread(new TimePrinter1(1000, "Fast Guy"));
        t1.start();
        Thread t2 = new Thread(new TimePrinter1(3000, "Slow Guy"));
        t2.start();
    }
}
```

适用于采用实现Runnable接口方法的情况

- ① 因为Java只允许单继承，如果一个类已经继承了Thread，就不能再继承其他类。
- ② 特别是在除了run()方法以外，并不打算重写Thread类的其它方法的情况下，以实现Runnable接口的方式生成新线程会更加合理。

Thread类的常用方法(1)

1. `start()`: 启动一个线程。
2. `run()`: 线程体,由`start()`方法调用,当`run()`方法返回时,当前的线程结束。
3. `currentThread()`: 返回当前运行的Thread对象。
4. `setName()`: 设置线程的名字。
5. `getName()`: 返回该线程的名字。
6. `setPriority()`: 设置线程优先级。
7. `getPriority()`: 返回线程优先级。
8. `stop()`: 使调用它的线程立即停止执行。
9. `isAlive()`: 如果线程已被启动并且未被终止,那么
`isAlive()`: 返回true。如果返回false,则该线程是新创建或是已被终止的。

Thread类的常用方法(1)

- 10. `sleep(int n)`: 使线程睡眠n毫秒,n毫秒后,线程可以再次运行。
- 11. `yield()`: 将CPU控制权主动移交到下一个可运行线程。
- 12. `join()`: 方法`join()`将引起现行线程等待,直至方法`join`所调用的线程结束。
- 13. `suspend()`: 使线程挂起,暂停运行。
- 14. `resume()` 恢复挂起的线程,使其处于可运行状态(Runnable)。
- 15. `wait()`:
- 16. `notify()`:
- 17. `notifyAll()`:

注意事项

- 因为Java线程的调度不是分时的，所以你必须确保你的代码中的线程会不时地给另外一个线程运行的机会。
- 有三种方法可以做到一点：
 - ①让处于运行状态的线程调用Thread.sleep()方法。
 - ②让处于运行状态的线程调用Thread.yield()方法。
 - ③让处于运行状态的线程调用另一个线程的join()方法。

sleep()和yield()的区别 (1)

- ① 这两个方法都是静态的实例方法。
- ② sleep()使线程转入阻塞状态，而yield()使线程转入runnable状态。
- ③ yield()给相同优先级或更高的线程运行机会，如果当前没有存在相同优先级的线程，则yield()什么都不做。而sleep()不会考虑线程的优先级，会给其他线程运行的机会，因此也会给相同或更低优先级线程运行机会。

sleep()和yield()的区别 (2)

- ④ `sleep()`会有中断异常抛出，而`yiled()`不抛出任何异常。
- ⑤ `sleep()`方法具有更好的可移植性，因为`yield()`的实现还取决于底层的操作系统对线程的调度策略。
- ⑥ 对于`yield()`的主要用途是在测试阶段，人为的提高程序的并发性能，以帮助发现一些隐藏的并发错误，当程序正常运行时，则不能依靠`yield`方法提高程序的并发性能。

sleep()方法

- sleep()有两个重载方法：
 - sleep(long mills)
 - sleep(long mills,int nanos)
- TimeUnit对sleep()方法提供了很好的封装，建议使用TimeUnit.XXXX.sleep()去代替Thread.sleep():
 - TimeUnit.SECONDS.sleep(1);
 - TimeUnit.MINUTES.sleep(3);

示例代码

```
public void run() {  
    while (true) {  
        try {  
            System.out.println("Printer " + printerId + " is running at " +  
                new Date(System.currentTimeMillis()));  
            Thread.yield();  
            // Thread.sleep(pauseTime);  
        } catch (Exception e) {  
            e.printStackTrace();  
        }  
    }  
}
```

Run | Debug

```
public static void  
    TimePrinter tp1  
    TimePrinter tp2  
    tp1.start();  
    tp2.start();  
}
```

```
Printer 1 is running at Wed Dec 07 20:59:05 CST 2022  
Printer 2 is running at Wed Dec 07 20:59:05 CST 2022  
Printer 1 is running at Wed Dec 07 20:59:05 CST 2022  
Printer 2 is running at Wed Dec 07 20:59:05 CST 2022  
Printer 1 is running at Wed Dec 07 20:59:05 CST 2022  
Printer 2 is running at Wed Dec 07 20:59:05 CST 2022  
Printer 1 is running at Wed Dec 07 20:59:05 CST 2022  
Printer 2 is running at Wed Dec 07 20:59:05 CST 2022  
Printer 1 is running at Wed Dec 07 20:59:05 CST 2022  
Printer 2 is running at Wed Dec 07 20:59:05 CST 2022
```

join() 方法:

- 作用: 使当前正在运行的线程暂停下来, 等待指定的时间后或等待调用该方法的线程结束后, 再恢复运行

1. `public final void join() throws InterruptedException;`
2. `public final void join(long millis, int nanos) throws InterruptedException;`
3. `public final void join(long millis) throws InterruptedException;`

```
public class TestSubThread3{
    public static void main(String args[]){
        SubThread t1=new SubThread("线程_1");
        SubThread t2=new SubThread("线程_2");
        t1.start();
        t2.start();
        System.out.println("main()方法运行完毕!");
    }
}

class SubThread extends Thread{
    String str;
    public SubThread(String s){
        str=s;
    }
    public void run(){
        for(int i=1;i<=3;i++){
            for(long j=1;j<1000000000;j++)
                ;
            System.out.println(str+": 第"+i+"次运行");
        }
        System.out.println(str+": 结束!");
    }
}
```

主线程和其他
线程共存

运行结果

```
main()方法运行完毕!  
线程_2: 第1次运行  
线程_1: 第1次运行  
线程_2: 第2次运行  
线程_1: 第2次运行  
线程_1: 第3次运行  
线程_1: 结束!  
线程_2: 第3次运行  
线程_2: 结束!
```

示例代码

```
public class TestSubThread1{
    public static void main(String[] args)throws InterruptedException{
        SubThread t1=new SubThread("First 线程");
        SubThread t2=new SubThread("Second 线程");
        t1.start();
        t2.start();
        Thread.sleep(5000);
        System.out.println("Main()方法运行完毕! ");
    }
}
class SubThread extends Thread{
    String str;
    public SubThread(String s){
        str=s;
    }
    public void run(){
        for(int i=1;i<=3;i++){
            for(long j=1;j<1000000000;j++)
                ;
            System.out.println(str+": 第"+i+"次运行");
        }
        System.out.println(str+": 结束! ");
    }
}
```

Thread.sleep()方法的使用，注意异常的抛出

运行结果

```
First 线程: 第1次运行  
Second 线程: 第1次运行  
main()方法运行完毕!  
First 线程: 第2次运行  
Second 线程: 第2次运行  
First 线程: 第3次运行  
First 线程: 结束!  
Second 线程: 第3次运行  
Second 线程: 结束!
```

示例代码

```
public class TestSubThread{  
    public static void main(String args[]) {  
        SubThread t1=new SubThread("线程_1");  
        SubThread t2=new SubThread("线程_2");  
        t1.start();  
        t2.start();  
        try{  
            t1.join();  
            t2.join();  
        }  
        catch (InterruptedException e) {  
            System.out.println(e.toString());  
        }  
        System.out.println("main()方法运行完毕！");  
    }  
}
```

join()方法的使用，注意异常的抛出。保证了主线程最后结束

```

class SubThread extends Thread{
    String str;
    public SubThread(String s){
        str=s;
    }
    public void run(){
        for(int i=1;i<=3;i++){
            for(long j=1;j<1000000000;j++)
                ;
            System.out.println(str+"： 第"+i+"次运行");
        }
        System.out.println(str+"： 结束！");
    }
}

```

```

线程_2: 第1次运行
线程_1: 第1次运行
线程_2: 第2次运行
线程_1: 第2次运行
线程_2: 第3次运行
线程_2: 结束!
线程_1: 第3次运行
线程_1: 结束!
main()方法运行完毕!

```

终止线程

- 当线程执行完`run()`方法，它将自然终止运行。
- `Thread`有一个`stop()`方法，可以强制结束线程，但这种方法是不安全的。因此，`stop()`方法已经被废弃。
- 实际编程中，一般是定义一个标志变量，然后通过程序来改变标志变量的值，从而控制线程从`run()`方法中自然退出。

示例代码

```
public class MyThreadStop extends Thread{
    int a; boolean flag=false;
    public void run(){
        while(!flag){

            System.out.println(a++);
        }
    }
    public void setFlag(boolean _flag){
        flag=_flag;
    }
    public static void main(String args[]){
        MyThreadStop t=new MyThreadStop();
        t.start();
        try{ Thread.sleep(1000);}catch(Exception e){}
        t.setFlag(true);
    }
}
```

设置线程优先级

```
public class ThreadPri
{
    public static void main(String args[ ])
    {
        MyThread th[ ]=new MyThread[3];
        for(int i=0;i<3;i++){
            th[i]=new MyThread( );
            th[i].start( );
        }
        th[0].setPriority(Thread.MAX_PRIORITY);
        //th[1].setPriority(Thread.NORM_PRIORITY);
        th[2].setPriority(Thread.MIN_PRIORITY);
    }
}

class MyThread extends Thread
{
    public void run( )
    {
        for(int i=0;i<2;i++)
            System.out.println(getName( )+"线程的优先级是: \t"+getPriority( )+"。");
    }
}
```

```
Thread-1线程的优先级是: 10。
Thread-1线程的优先级是: 10。
Thread-2线程的优先级是: 5。
Thread-2线程的优先级是: 5。
Thread-3线程的优先级是: 1。
Thread-3线程的优先级是: 1。
```

主要内容

- 进程与线程
- 线程的调度
- Java多线程程序设计
 - 创建和启动线程
 - Thread线程类
 - Runnable接口
 - 多个线程的同步
 - 线程之间的通信

- 在多线程环境中, 两个或者两个以上的线程访问一个共享对象的情况是很普遍的
- 比如某公司在银行里有一个数据库, 其在北京和上海的营业部都可以使用, 那么如果北京的营业部在存款的时候, 上海的营业部正好在取款, 会发生什么事呢?

- 线程安全是并发编程中的重要关注点，造成线程安全问题的主要诱因有两点：
 - 一是存在共享数据(也称临界资源)
 - 二是存在多条线程共同操作共享数据

共享数据的线程安全问题怎么处理？

- 自然而然的想法就是每一个线程依次去读写这个共享变量，这样就不会有任何数据安全的问题，因为每个线程所操作的都是当前最新的版本数据。
- Java关键字synchronized就具有使每个线程依次排队操作共享变量的功能。

示例代码: ThreadDemo2.java

```
public class ThreadDemo2 {
```

Run | Debug

```
    public static void main(String[] args) {
```

```
        Runnable r1 = new TestThread2();
```

```
        new Thread(r1).start();
```

```
        new Thread(r1).start();
```

```
        new Thread(r1).start();
```

```
        new Thread(r1).start();
```

```
    }
```

```
}
```

```
class TestThread2 implements Runnable {
```

```
    int count = 100;
```

```
    public void run() {
```

```
        try {
```

```
            while(count>0){
```

```
                Thread.sleep(10);
```

```
                System.out.println(Thread.currentThread().getName() +  
                                     " is sailing ticket" + count);
```

```
                count--;
```

```
            }
```

```
        } catch (Exception e) {
```

```
            e.printStackTrace();
```

```
        }
```

```
    }
```

运行结果

```
Thread-1 is sailing ticket17
Thread-0 is sailing ticket16
Thread-2 is sailing ticket15
Thread-3 is sailing ticket14
Thread-1 is sailing ticket13
Thread-0 is sailing ticket12
Thread-2 is sailing ticket11
Thread-3 is sailing ticket10
Thread-1 is sailing ticket9
Thread-0 is sailing ticket8
Thread-2 is sailing ticket7
Thread-3 is sailing ticket7
Thread-0 is sailing ticket5
Thread-1 is sailing ticket5
Thread-3 is sailing ticket3
Thread-2 is sailing ticket2
Thread-0 is sailing ticket1
Thread-1 is sailing ticket0
Thread-3 is sailing ticket-1
Thread-2 is sailing ticket-2
```

线程的互斥锁

- 同步是一种保证共享资源完整性的手段；
- 具体作法是在代表原子操作的程序代码前加上synchronized标记，这样的代码被称为同步代码块。
- Java中用关键字synchronized为共享资源加锁，在任何一个时刻只能有一个线程能取得加锁对象的钥匙，对加锁对象进行操作。从而达到了对共享资源访问时的保护。

关键字synchronized的使用方法(1)

- `public synchronized void run();`
 - 一个线程调用某个对象的synchronized方法, 就锁定了这个对象, 直到它从这个方法中退出, 才释放了这个锁定。因此, 一旦一个对象被某个线程锁定了, 它的**所有synchronized**方法就都不允许别的线程调用了。
 - 如果一个线程调用该方法, 则方法体中访问的对象都被锁定, 不允许其他线程访问, 以解决共享资源的访问冲突问题。

示例代码: ThreadDemo2.java

```
public class ThreadDemo2 {  
    Run | Debug  
    public static void main(String[] args) {  
        Runnable r1 = new TestThread2();  
        new Thread(r1).start();  
        new Thread(r1).start();  
        new Thread(r1).start();  
        new Thread(r1).start();  
    }  
}  
  
class TestThread2 implements Runnable {  
    int count = 100;  
  
    public synchronized void run() {  
        try {  
            while(count>0){  
                Thread.sleep(10);  
                System.out.println(Thread.currentThread().getName() +  
                    " is sailing ticket" + count);  
                count--;  
            }  
        } catch (Exception e) {  
            e.printStackTrace();  
        }  
    }  
}
```

运行结果

```
Thread-0 is sailing ticket27  
Thread-0 is sailing ticket26  
Thread-0 is sailing ticket25  
Thread-0 is sailing ticket24  
Thread-0 is sailing ticket23  
Thread-0 is sailing ticket22  
Thread-0 is sailing ticket21  
Thread-0 is sailing ticket20  
Thread-0 is sailing ticket19  
Thread-0 is sailing ticket18  
Thread-0 is sailing ticket17  
Thread-0 is sailing ticket16  
Thread-0 is sailing ticket15  
Thread-0 is sailing ticket14  
Thread-0 is sailing ticket13  
Thread-0 is sailing ticket12  
Thread-0 is sailing ticket11  
Thread-0 is sailing ticket10  
Thread-0 is sailing ticket9  
Thread-0 is sailing ticket8  
Thread-0 is sailing ticket7  
Thread-0 is sailing ticket6  
Thread-0 is sailing ticket5  
Thread-0 is sailing ticket4  
Thread-0 is sailing ticket3  
Thread-0 is sailing ticket2  
Thread-0 is sailing ticket1
```

关键字synchronized的使用方法(2)

- `synchronized (obj) { ... }`
- 一个线程进入由`synchronized (obj) { ... }`修饰的程序块, 就锁定了由`obj`指定的那个对象。

示例代码: ThreadDemo1.java

```
public static void main(String[] args) {  
    Runnable r1 = new TestThread2();  
    new Thread(r1).start();  
    new Thread(r1).start();  
    new Thread(r1).start();  
    new Thread(r1).start();  
}  
}  
  
class TestThread2 implements Runnable {  
    int count = 100;  
    String a = "hello";  
  
    public void run() {  
        try {  
            synchronized(a){  
                while(count>0){  
                    Thread.sleep(10);  
                    System.out.println(Thread.currentThread().getName() +  
                        " is sailing ticket" + count);  
                    count--;  
                }  
            }  
        } catch (Exception e) {  
            e.printStackTrace();  
        }  
    }  
}
```

运行结果

```
Thread-0 is sailing ticket27
Thread-0 is sailing ticket26
Thread-0 is sailing ticket25
Thread-0 is sailing ticket24
Thread-0 is sailing ticket23
Thread-0 is sailing ticket22
Thread-0 is sailing ticket21
Thread-0 is sailing ticket20
Thread-0 is sailing ticket19
Thread-0 is sailing ticket18
Thread-0 is sailing ticket17
Thread-0 is sailing ticket16
Thread-0 is sailing ticket15
Thread-0 is sailing ticket14
Thread-0 is sailing ticket13
Thread-0 is sailing ticket12
Thread-0 is sailing ticket11
Thread-0 is sailing ticket10
Thread-0 is sailing ticket9
Thread-0 is sailing ticket8
Thread-0 is sailing ticket7
Thread-0 is sailing ticket6
Thread-0 is sailing ticket5
Thread-0 is sailing ticket4
Thread-0 is sailing ticket3
Thread-0 is sailing ticket2
Thread-0 is sailing ticket1
```

关键字synchronized的使用方法(3)

- `public static synchronized int getValue();`
- 一个线程调用一个类的static synchronized方法, 就锁定了这个类对象, 也就控制了所有static synchronized方法。即类中所有的静态同步方法在每个时刻只有一个可以执行

注意事项

- ① 一个对象在某一时刻只能被一个线程锁定；
- ② 一个类可以有多个对象, 不同的对象可以被不同的线程锁定
- ③ 一个线程可以锁定多个对象

误区

- 关键字synchronized可以作为Java方法修饰符，也可以作为Java方法内的语句。
- 被它修饰的代码部分往往被描述为临界区。这使很多人认为，由于代码被synchronized保护着，因此同一时刻只能有一个线程访问它。但这种观点是错误的。

- 对于Java类中的方法，关键字 `synchronized` 其实并不锁定该方法或该方法的部分代码，它只是锁定对象。
- `synchronized` 方法或 `synchronized` 区段内的代码在同一时刻下可有多个线程执行，只要是对不同的对象调用该方法就可以。

主要内容

- 进程与线程
- 线程的调度
- Java多线程程序设计
 - 创建和启动线程
 - Thread线程类
 - Runnable接口
 - 多个线程的同步
 - 线程之间的通信

- 不同的线程执行不同的任务，如果这些任务有某种联系，线程之间必须能够通信，协调完成工作。
- 例如生产者和消费者共同操作堆栈，当堆栈为空时，消费者无法取出产品，应该先通知生产者向堆栈中加入产品。当堆栈已满时，生产者无法继续加入产品，应该先通知消费者从堆栈中取出产品。

☛ java.lang.Object 类中提供了两个用于线程通信的方法：

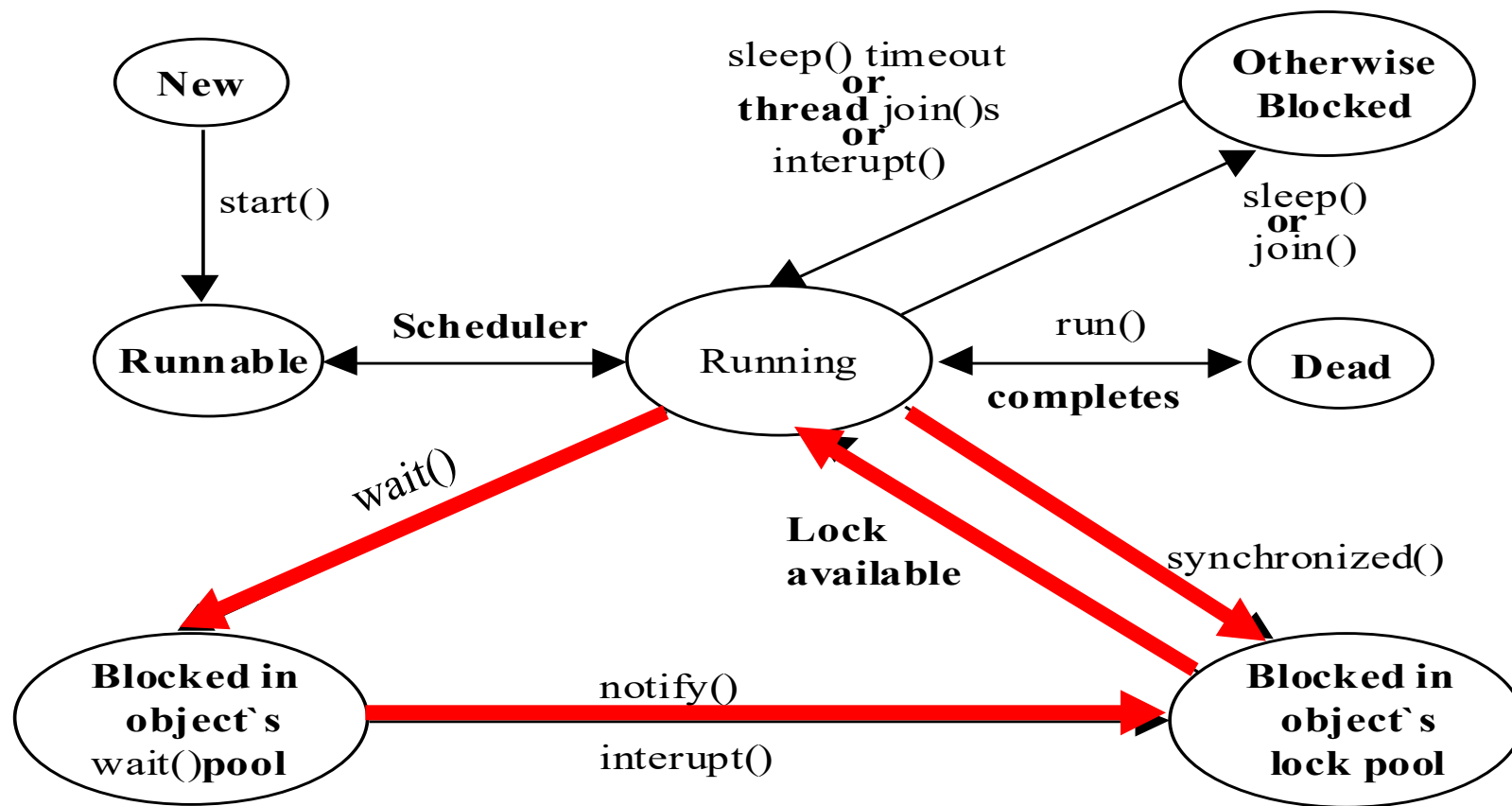
- wait()：执行该方法的线程释放对象的锁，Java 虚拟机把该线程放到该对象的等待池中。该线程等待其他线程将它唤醒。
- notify()：执行该方法的线程唤醒在对象的等待池中等待的一个线程。Java 虚拟机从对象的等待池中随机的选择一个线程，把它转到对象的锁池中。

线程状态说明

- 一般来说，每个共享对象的互斥锁存在两个队列，一个是锁等待队列（等待池），另一个是锁申请队列（锁池），锁申请队列中的第一个线程可以对该共享对象进行操作，而锁等待队列中的线程在某些情况下将移入到锁申请队列。

线程状态

Thread states



wait()、notify()和notifyAll()方法

- wait, notify, notifyAll 必须在已经持有锁的情况下执行, 所以它们只能出现在 synchronized 作用的范围内, 也就是出现在 synchronized 修饰的方法中。
- wait 的作用: 释放已持有的锁, 进入等待队列。
- notify 的作用: 随机唤醒 wait 队列中的一个线程并把它移入锁申请队列
- notifyAll 的作用: 唤醒 wait 队列中的所有线程并把它它们移入锁申请队列。

wait()、notify()和notifyAll()方法

- 一个“在同步方法中等待”的线程自己是没有办法恢复运行的, 只能等另一个也进入了该对象的synchronized方法的线程调用notify和notifyAll后才有可能恢复运行

方法wait()与sleep() 方法对比

- ① 方法wait()与sleep() 方法一样，都能使线程等待而停止运行
- ② 其区别在于sleep()方法不会释放对象的锁，而wait()方法进入等待时，可以释放对象的锁，因而别的线程能对这些加锁的对象进行操作。所以，wait, notify和notifyAll都是与同步相关联的方法，只有在synchronized方法中才可以用。
- ③ 在不同步的方法或代码中则使用sleep()方法使线程暂时停止运行。

案例：PV操作

- 多个生产者生产产品，多个消费者消费产品，产品要提供一个含有标识的id属性，另外需要在生产或消费时打印产品的详细内容；
- 店员一次最多只能持有10份产品，如果生产者生产的产品多于10份，则会让生产者线程等待；
- 当店员没有产品消费时，消费线程等待；

- 多个生产者线程和多个消费者线程都需要面向店员，生产者线程生产的产品交给店员消费，消费者线程从店员那里获得产品消费；
- 店员在操作产品时，需要做到互斥；
- 产品类：Products
- 店员类：Clerk
- 生产者线程：Producer
- 消费者线程：Consumer

```
public class Products {  
    int id;  
    public Products() {  
        super();  
    }  
    public Products(int id) {  
        super();  
        this.id = id;  
    }  
    public String toString() {  
        return "Products [id=" + id + "]";  
    }  
}
```

```
public class Clerk {  
    int index = 0; // 默认为0个产品  
    Products[] pro = new Products[10];  
    // 生产者生产出来的产品交给店员  
    public synchronized void addProduct(Products pd) {  
        while (index == pro.length) {  
            try {  
                this.wait(); // 产品已满，请稍后再生产  
            } catch (InterruptedException e) {  
                e.printStackTrace();  
            }  
        }  
        this.notify(); // 通知等待区的消费者可以取产品了  
        pro[index] = pd;  
        index++;  
    }  
}
```

// 消费者从店员处取产品

```
public synchronized Products getProduct() {  
    while (index == 0) {  
        try {  
            this.wait(); // 缺货，请稍后再取。  
        } catch (InterruptedException e) {  
            e.printStackTrace();  
        }  
    }  
    this.notify(); // 通知等待区的生产者可以生产产品了  
    index--;  
    return pro[index];  
}
```

```
}
```

```

public class Producer implements Runnable {
    private Clerk clerk;
    public Producer() {
        super();
    }
    public Producer(Clerk clerk) {
        super();
        this.clerk = clerk;
    }
    public void run() {
        System.out.println("生产者开始生产产品");
        for (int i = 0; i < 15; i++) {
            // 注意此处的循环次数一定要大于pro数组的长度(10)
            Products pd = new Products(i);
            clerk.addProduct(pd); // 生产产品
            System.out.println("生产了: " + pd);
            try { // 睡眠时间用随机产生的值来设置
                Thread.sleep((int) (Math.random() * 10) * 100);
            } catch (InterruptedException e) {
                e.printStackTrace();
            }
        }
    }
}

```

```
public class Consumer implements Runnable {
    private Clerk clerk;
    public Consumer() {
        super();
    }
    public Consumer(Clerk clerk) {
        super();
        this.clerk = clerk;
    }
    public void run() {
        System.out.println("消费者开始取走产品");
        for (int i = 0; i < 15; i++) {
            // 注意此处的循环次数一定要大于pro数组的长度(10)
            // 取产品
            Products pd = clerk.getProduct();
            System.out.println("消费了: " + pd);
            try { // 睡眠时间用随机产生的值来设置
                Thread.sleep((int) (Math.random() * 10) * 100);
            } catch (InterruptedException e) {
                e.printStackTrace();
            }
        }
    }
}
```

```
public class ProducerAndConsumerTest {  
    public static void main(String[] args) {  
        Clerk clerk = new Clerk();  
        Thread producerThread = new Thread(new Producer(clerk));  
        Thread consumerThread = new Thread(new Consumer(clerk));  
        Thread producerThread1 = new Thread(new Producer(clerk));  
        Thread consumerThread2 = new Thread(new Consumer(clerk));  
        producerThread.start();  
        consumerThread.start();  
        producerThread1.start();  
        consumerThread2.start();  
    }  
}
```

Partial results

```
消费了: Products [id=4]  
生产了: Products [id=3]  
消费了: Products [id=3]  
消费了: Products [id=2]  
消费了: Products [id=1]  
生产了: Products [id=5]  
消费了: Products [id=5]  
生产了: Products [id=4]  
消费了: Products [id=4]
```

小结

- 线程之间交换信息称为线程通信，可以认为，wait()和notify()方法是java采用的一种简单的线程间通信机制，利用它们，彼此间只传送一个信号。
- 线程间要传送的数据较多时，必须使用其它的像共享主存、管道流等通信方式。
 - ① 主存读/写通信:Java输入输出流
 - ② 管道流通信:

- 管道用来把一个程序输出连接到另一程序输入。java.io中提供了类PipedInputStream和PipedOutputStream作为管道的输入/输出部件。
 - 构造方法中，对于管道输入/输出流来说，对应管道输出/输入流作为参数以进行连接；
 - 管道输入/输出流还提供了方法connect()以进行相应的连接。

```
import java.io.*;
import java.util.*;

public class TestPipedStream {
    public static void main(String[] args) {
        try {
            PipedOutputStream out = new PipedOutputStream();
            PipedInputStream in = new PipedInputStream();
            out.connect(in);
            ThreadOut to = new ThreadOut(out);
            ThreadIn ti = new ThreadIn(in);
            to.start();
            ti.start();
        } catch (IOException e) {
            System.out.println(e);
        }
    }
}
```

```

class ThreadOut extends Thread {
    private int i1 = 12345;
    private boolean b1 = true;
    private char c1 = 'a';
    private String s1 = "Java 程序设计";
    private DataOutputStream dos;

    public ThreadOut(PipedOutputStream out) {
        dos = new DataOutputStream(out);
    }

    public void run() {
        System.out.println("输出管道发送以数据");
        System.out.println("\t整数: " + i1 + "\t逻辑值: " + b1 + "\t字符: " + c1
            + "\t字符串: " + s1);

        try {
            dos.writeInt(i1);
            dos.writeBoolean(b1);
            dos.writeChar(c1);
            dos.writeUTF(s1);
            dos.close();
        } catch (IOException e) {
        }
    }
}

```

```

class ThreadIn extends Thread {
    private DataInputStream dis;

    public ThreadIn(PipedInputStream in) {
        dis = new DataInputStream(in);
    }

    public void run() {
        try {
            int i2 = dis.readInt();
            boolean b2 = dis.readBoolean();
            char c2 = dis.readChar();
            String s2 = dis.readUTF();
            System.out.println("这是输入管道接收到的输出管道发送的数据: ");
            System.out.println("\t整数: " + i2 + "\t逻辑值: " + b2 + "\t字符: " + c2
                               + "\t字符串" + s2);

            dis.close();
        } catch (IOException e) {
            System.out.println(e);
        }
    }
}

```

死锁 (1)

- ✓ 在多线程竞争使用多资源的程序中，有可能出现死锁的情况。死锁不是由于资源短缺造成的，而是由于程序设计不合理而造成程序中所有的线程都陷入等待态或阻塞态。
- ✓ 这种情况发生在一个线程等待另一个线程所持有的锁，而那个线程又在等待第一个线程持有的锁的时候。

死锁 (2)

- ✓ 每个线程都不能继续运行，除非另一线程运行完同步程序块。
- ✓ 因为哪个线程都不能继续运行，所以哪个线程都无法运行完同步程序块。
- ✓ 另一种情况是在wait, notify结构中, 如果所有的线程都wait了, 就不会有线程来notify它们, 因此要特别注意;

死锁

```
public void run() {  
    if(flag) {  
        while(true) {  
            synchronized(MyLock.obj1) {  
                System.out.println(Thread.currentThread().getName()+"....i  
                synchronized(MyLock.obj2) {  
                    System.out.println(Thread.currentThread().getName()+".  
                }  
            }  
        }  
    }  
    else {  
        while(true){  
            synchronized(MyLock.obj2) {  
                System.out.println(Thread.currentThread().getName()+"....e  
                synchronized(MyLock.obj1) {  
                    System.out.println(Thread.currentThread().getName()+".  
                }  
            }  
        }  
    }  
}
```

小结 (1)

- 可以采用 Thread 类的子类或实现 Runnable 接口，实现多线程
- 两个关键性操作：
 - 定义用户线程的操作，即定义用户线程的run()方法；
 - 在适当的时候建立用户线程的实例。

小结 (2)

- 继承 Thread 类

- 用户创建自己的 Thread 类的子类，并在子类中重新定义自己的 run() 方法，run() 方法中包含了用户线程的操作。
- 在需要建立自己的线程时，只需创建一个已定义好的 Thread 子类对象就可以了。

小结 (3)

- 实现 Runnable 接口
 - 编写实现 Runnable 接口的线程类，重写 Runnable 接口中的 run() 方法。
 - 在实现 Runnable 接口的同时还可以继承其它类。

小结（4）

- Java中用关键字 **synchronized** 为共享资源加锁，在任何一个时刻只能有一个线程能取得加锁对象的钥匙，对加锁对象进行操作。从而达到了对共享资源访问时的保护。
- synchronized关键字可以使用在：
 1. 一个成员方法上
 2. 一个静态方法上
 3. 一个语句块上

小结 (5)

- 线程间传递信号：
 - wait, notify和notifyAll都是与同步相关联的方法, 只有在synchronized方法中才可以用
- 主存读/写通信
- 管道流通信

以下内容了解即可



2023/12/12



北京航空航天大学
COLLEGE OF SOFTWARE
BEIHANG UNIVERSITY 软件学院

后台线程

- 后台线程是指为其他线程提供服务的线程，也称为守护线程（**Daemon Threads**）。
- 守护线程是一类特殊的线程，它和普通线程的区别在于它并不是应用程序的核心部分，当一个应用程序的所有非守护线程终止运行时，**即使仍然有守护线程在运行，应用程序也将终止。**
- **反之，只要有一个非守护线程在运行，应用程序就不会终止。**
- 主线程默认情况下是前台线程，由前台线程创建的线程默认情况下也是前台线程。



守护线程(Daemon Threads)

- 可以通过调用方法 `isDaemon()` 来判断一个线程是否是守护线程
- 程，也可以调用方法 `setDaemon()` 来将一个线程设为守护线程。
 - *Garbage collector is a daemon thread*

守护线程(Daemon Threads)

```
public class Machine extends Thread {  
    private int a;  
    private static int count;  
    public void start() {  
        super.start();  
        // 匿名线程类  
        Thread daemon = new Thread() {  
            public void run() {  
                while (true) { // 无限循环  
                    reset();  
                    try {  
                        sleep(50);  
                    } catch (InterruptedException e) {  
                        throw new RuntimeException(e);  
                    }  
                }  
            }  
        };  
        daemon.setDaemon(true);  
        daemon.start();  
    }  
}
```

```
public void reset() {  
    a = 0;  
}  
public void run() {  
    while (true) {  
        System.out.println(getName() + ":" + a++);  
        if (count++ == 100)  
            break;  
        yield();  
    }  
}  
public static void main(String args[]) throws Exception {  
    Machine machine = new Machine();  
    machine.start();  
}  
}
```

定时器Timer

- 在JDK的java.util包中提供了一个实用类Timer，它能够定时执行特定的任务。
- TimerTask类表示定时器执行的一项任务。
- Timer类的`schedule(TimerTask task, long delay, long period)`方法用来设置定时器需要定时执行的任务。task参数表示任务，delay参数表示延迟执行的时间，以毫秒为单位，period参数表示每次执行任务的间隔时间，以毫秒为单位。



定时器

- `timer.schedule(task,10,50);`
- 以上代码表明定时器将在10毫秒以后开始执行task任务（即执行TimerTask实例的run()方法），以后每隔50毫秒重复执行一次task任务。



定时器

```
import java.util.Timer;
import java.util.TimerTask;
public class Machine1 extends Thread {
    private int a;
    public void start() {
        super.start();
        // 把与Timer关联的线程设为后台线程
        Timer timer = new Timer(true);
        // 匿名类
        TimerTask task = new TimerTask() {
            public void run() {
                reset();
            }
        };
        // 设置定时任务
        timer.schedule(task, 10, 50);
    }
}
```



```
public void reset() {  
    a = 0;  
}  
  
public void run() {  
    for (int i = 0; i < 1000; i++) {  
        System.out.println(getName() + ":" + a++);  
        yield();  
    }  
}  
  
public static void main(String args[]) throws Exception {  
    Machine1 machine = new Machine1();  
    machine.start();  
}  
}
```

思考：多线程问题

- 对多线程程序本身来说，它会对系统产生以下影响：
 - 线程需要占用内存。
 - 线程过多，会消耗大量CPU时间来跟踪线程。
 - 必须考虑多线程同时访问共享资源的问题，如果没有协调好，就会产生令人意想不到的问题，例如可怕的死锁和资源竞争。
 - 因为同一个任务的所有线程都共享相同的地址空间，并共享任务的全局变量，所以程序也必须考虑多线程同时访问全局变量的问题。

Callable接口(了解)

```
public interface Runnable {  
    public abstract void run();  
}
```

- Java中实现多线程有三种方法：
 - 一种是继承Thread类；
 - 第二种是实现Runnable接口；
 - 第三种是实现Callable接口；
 - Callable接口也是位于java.util.concurrent包中。Callable接口的定义为：

```
public interface Callable<V> {  
    V call() throws Exception;  
}
```

```
package com.buaa.test;
import java.util.ArrayList;
import java.util.concurrent.Callable;
import java.util.concurrent.ExecutionException;
import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;
import java.util.concurrent.Future;
import java.util.concurrent.TimeUnit;

class TaskWithResult implements Callable<String>{
    private int id;
    private static int count =10;
    private final int time =count--;
    public TaskWithResult(int id){
        this.id = id;
    }

    @Override
    public String call() throws Exception {
        TimeUnit.MILLISECONDS.sleep(100);
        return "Result of TaskWithResult : "+ id+", Time= "+time;
    }
}
```

```
public class CallableDemo {  
    public static void main(String[] args)  
        throws InterruptedException, ExecutionException {  
        ExecutorService exec = Executors.newCachedThreadPool();  
        ArrayList<Future<String>> results = new ArrayList<Future<String>>();  
        for(int i=0;i<10;i++){  
            results.add(exec.submit(new TaskWithResult(i)));  
        }  
  
        for(Future<String> fs : results){  
            System.out.println(fs.get());  
        }  
    }  
}
```

```
Result of TaskWithResult : 0, Time= 10
Result of TaskWithResult : 1, Time= 9
Result of TaskWithResult : 2, Time= 8
Result of TaskWithResult : 3, Time= 7
Result of TaskWithResult : 4, Time= 6
Result of TaskWithResult : 5, Time= 5
Result of TaskWithResult : 6, Time= 4
Result of TaskWithResult : 7, Time= 3
Result of TaskWithResult : 8, Time= 2
Result of TaskWithResult : 9, Time= 1
```

Callable 和 Runnable 的使用方法对比

- 1、Callable 使用 `call()` 方法，Runnable 使用 `run()` 方法
- 2、Callable的任务执行后可返回值，而Runnable的任务不能有返回值(是void)
- 3、`call()` 可以抛出受检查的异常，而`run()`不能抛出受检查的异常。
- 4、运行Callable任务可以拿到一个Future对象，表示异步计算的结果。它提供了检查计算是否完成的方法，以等待计算的完成，并检索计算的结果。通过Future对象可以了解任务执行情况，可取消任务的执行，还可获取执行结果。

Executor框架

